

DECK 06 • FINAL WEEK

Build your capstone.

One complete project, built from what you
learned in weeks 2-5. Direct, practical, shippable.

NO PRIZES. NO JUDGING. REAL OUTPUT.

The capstone is your portfolio piece.

- 01 Combine skills from at least 3 previous exercises.
- 02 Ship one complete project, not disconnected demos.
- 03 Prefer simple and working over ambitious and broken.



PORTFOLIO FIRST

Your project is on devnet. Anyone can verify transactions, accounts, and behavior.

MINIMUM TECH BAR

Use at least three prior building blocks in one coherent product.

On-chain logic

Program constraints, PDA state, authority checks, and instruction flow.

Token operations

SPL token accounts, transfers, or Token-2022 features where relevant.

Composability

CPI patterns or integrations with existing protocols/APIs.

User interface

Working frontend or CLI that proves end-to-end usability.

TOKEN-GATED VOTING APP

Votes weighted by tokens.

NFT MARKETPLACE LISTING

List and buy NFTs atomically.

TREASURY MANAGER

Accept SOL and SPL deposits, track per-user balances, withdraw proportionally.

- 01 Multiple instructions for deposit/withdraw/reporting.
- 02 Vault accounting per depositor and per asset.
- 03 Frontend dashboard showing treasury state.



GOOD FIT IF

You want a project that mixes state machines, token handling, and practical portfolio utility.

NON-NEGOTIABLE OUTPUTS

Ship these three things. No extras needed.

01

DEPLOYED PROGRAM

Live on devnet with a verifiable program ID and real transactions.

02

WORKING INTERFACE

Frontend or CLI that demonstrates real user flow against your deployed logic.

03

README

What it does, how to run it, known limitations, and what you would improve next.

IF YOU CAN'T EXPLAIN IT, YOU DON'T OWN IT.

README is required, not optional.

SECTION	WHAT TO INCLUDE	WHY IT MATTERS
Project summary	Problem, users, and what the app does.	Shows product clarity.
Architecture	Programs, PDAs, accounts, and instruction flow.	Shows technical understanding.
Runbook	Setup, deploy, run UI/CLI, and test steps.	Lets others reproduce it.
Trade-offs	What you simplified and what you would improve.	Shows engineering maturity.

SCOPE DOWN TO FINISH STRONG

Simple + shipped wins.



DO

Pick one core flow and make it reliable end to end.



DON'T

Add three extra features before your base flow is stable.



DO

Add tests for failure cases, not just happy path.



DON'T

Leave explanation to the last minute. Write README as you build.

EXECUTION PLAN

Follow this order to reduce rework.

- 01

Write one-page design note: accounts, PDAs, instructions, authority model.
-
- 02

Implement and test the core on-chain instruction flow.
-
- 03

Add one interface (frontend or CLI) to drive real transactions.
-
- 04

Harden failure paths and constraints.
-
- 05

Finalize README with architecture, run steps, and improvement notes.

OPTIONAL

Deploy to mainnet.

Real SOL. Real transactions. Real costs. If you do this, keep scope tight and protect keys carefully.



MAINNET CAUTION

Only deploy what you understand line by line. Audit authority paths and close-account flows first.

FINISH WITH A PROJECT YOU CAN SHOW

A shipped capstone beats an unfinished giant idea. Build it, document it, and make it verifiable.

What success looks like

Program deployed, interface works, README is clear, and behavior is verifiable on devnet.

What this proves

You can design, build, debug, and ship a complete Solana project from scratch.